SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	Sk. Nuha	**Reg. No.:**	192472242
Section / Batch:	CSE-AI	**Date:**	

Instructions to Students

- Answer ALL questions with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions	Marks
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1 – Q3	Q1 –40 Q2 –30 Q3 -30
	GRAND TOTAL:	**100 Marks**	

SECTION A – Pseudocode Writing Task

Q1. Hamiltonian Cycle Problem with Forbidden Vertices in Intermediate Positions Design a pseudocode using Backtracking to find a Hamiltonian cycle in which certain vertices are not allowed to appear in specified intermediate positions of the cycle. Clearly identify inputs (graph, restricted position rules) and output (valid Hamiltonian cycle or failure). Develop a step-by-step algorithm that extends the path one vertex at a time while checking adjacency, visited status, and position restrictions. Use loops, recursive calls, and conditional checks effectively during cycle construction. Apply pruning to discard partial paths that violate position rules or revisit vertices. Ensure the pseudocode is well-structured, readable, and easy to trace. Handle all possible cases and guarantee correctness of the final decision.

Q2. Hamiltonian Cycle Problem with Required Visit Order Write a pseudocode using Backtracking to find a Hamiltonian cycle in which a given subset of vertices must appear in a specified relative order. Clearly define inputs (graph, ordered required vertices) and output (valid Hamiltonian cycle or no solution). Design an algorithm that builds the cycle while tracking visited vertices and verifying the relative ordering condition. Use loops, recursion, and adjacency checks effectively during path extension. Apply pruning whenever a partial path can no longer satisfy the required ordering or legal cycle constraints. Present the pseudocode in a clean and logically organized format. Ensure completeness and correctness for all valid cycles satisfying the extra order condition.

Code of Conduct

I Sk.Nuha-192472242 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sk. Nuha

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Question No.	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)	Total Marks (Each – 50)
Q1						
Q2						
Q3						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Hamiltonian Cycle Problem with Forbidden Vertices in Intermediate Positions

Aim

To find a Hamiltonian Cycle using Backtracking such that certain vertices are not allowed to appear in specified intermediate positions of the cycle.

Problem Statement

Given a graph and a set of position restrictions, find a Hamiltonian Cycle that visits every vertex exactly once and returns to the starting vertex while ensuring forbidden vertices do not appear in restricted positions.

Input

- Graph G(V,E)
- Restricted Position Rules
    - Example: Vertex 3 cannot be at position 2

Output

- Valid Hamiltonian Cycle
- OR "No Solution"

Algorithm

1. Start with vertex 0.
2. Mark vertex 0 as visited.
3. Recursively add vertices to the path.
4. Before adding a vertex:
    - Check adjacency.
    - Check if already visited.
    - Check position restrictions.
5. If all vertices are included:
    - Verify last vertex connects to first vertex.
6. If valid, output cycle.
7. Otherwise backtrack.
8. If no cycle exists, print "No Solution".

Pseudocode

```
HAM_CYCLE(Graph, Restriction)

path[0] ← 0
visited[0] ← true

IF HAM_UTIL(1) = true
   PRINT path
ELSE
   PRINT "No Solution"
```

```
HAM_UTIL(pos)

IF pos = N
   IF Graph[path[N-1]][path[0]] = 1
      RETURN true
   ELSE
      RETURN false

FOR v ← 1 TO N-1

   IF isSafe(v, pos)

      path[pos] ← v
      visited[v] ← true

      IF HAM_UTIL(pos + 1)
         RETURN true

      visited[v] ← false

RETURN false

isSafe(v, pos)

IF Graph[path[pos-1]][v] = 0
   RETURN false

IF visited[v] = true
   RETURN false

IF v is forbidden at position pos
   RETURN false

RETURN true
```

Conclusion

Backtracking explores all possible Hamiltonian cycles while pruning paths that violate adjacency, visited status, or position restrictions. This guarantees correctness while reducing unnecessary exploration.

Q104. Hamiltonian Cycle Problem with Required Visit Order

Aim

To find a Hamiltonian Cycle using Backtracking such that a specified subset of vertices appears in a required relative order.

Problem Statement

Given a graph and an ordered list of required vertices, find a Hamiltonian Cycle where every vertex is visited exactly once and the required vertices appear in the specified order.

Input

- Graph G(V,E)
- Ordered Required Vertices
    - Example: [2, 5, 7]
    - Vertex 2 must appear before 5 and 5 before 7.

Output

- Valid Hamiltonian Cycle
- OR "No Solution"

Algorithm

1. Start from vertex 0.
2. Mark it as visited.
3. Recursively extend the path.
4. Before adding a vertex:
    - Check adjacency.
    - Check visited status.
    - Verify ordering constraints.
5. If ordering cannot be satisfied, prune the path.
6. When all vertices are included:
    - Verify cycle closure.
7. Output cycle if valid.

Pseudocode

```
HAM_CYCLE_ORDER(Graph, OrderList)

path[0] ← 0
visited[0] ← true

IF HAM_UTIL(1)
   PRINT path
ELSE
   PRINT "No Solution"
```

```
HAM_UTIL(pos)

IF pos = N

   IF Graph[path[N-1]][path[0]] = 1
     AND OrderSatisfied(path)
      RETURN true

   RETURN false

FOR v ← 1 TO N-1

   IF isSafe(v, pos)

      path[pos] ← v
      visited[v] ← true

      IF OrderPossible(path, pos)

         IF HAM_UTIL(pos + 1)
            RETURN true

      visited[v] ← false

RETURN false

isSafe(v, pos)

IF Graph[path[pos-1]][v] = 0
   RETURN false

IF visited[v] = true
   RETURN false

RETURN true

OrderPossible(path, pos)

Check whether required order
can still be achieved.

IF violated
   RETURN false

RETURN true
```

Conclusion

The Backtracking algorithm constructs the Hamiltonian Cycle while maintaining the required vertex order. Early pruning eliminates paths that cannot satisfy the ordering constraint, improving efficiency while ensuring a correct solution.